

Sensors, Signals, and Data Processing

Prepared By

Dr. Islam Zakaria

Lecturer at the Information Technology Department, FCAIH, Capital University

Learning Objectives

- ❑ **Objective:** Managing the transition from analog physical phenomena to digital data streams.
- ❑ **CS Philosophy:** All sensors lie. The role of the software is to filter, calibrate, and fuse these "lies" to create a reliable model of the world.

From Physics to Variables

❑ The Golden Rule of Sensor-Based Computing:

"All sensors lie. The job of the software is to filter, calibrate, and combine these 'lies' into a reliable model of the world."

❑ Why this matters:

- A sensor does not know it is in a robot.
- It does not know what you are trying to measure.
- It only knows voltage.
- You must extract meaning from that voltage.

The Sensor-to-Software Pipeline

❑ The Chain of Truth:

Computer Scientists (CS) and Electrical Engineers (EE)

Stage	Description	CS/EE Responsibility
Physical Phenomenon	Heat, light, acceleration, distance	Understanding physics limits
Transducer	Converts physics → Voltage	EE: Circuit design
Analog Signal	Continuous voltage	EE: Amplification, shielding
ADC (Analog-to-Digital Converter)	Discrete quantization	CS: Understanding resolution limits
Binary Data	Bits (e.g., 0x3FF)	CS: Endianness, alignment
Software Variable	float temperature;	CS: Calibration, filtering

"Endianness" refers to what order the bytes are sent in

Data Formats - The Common Language of Robots

❑ Scalars (1 value per sample):

- Temperature, battery voltage, single-axis force (**Memory:** float or uint16_t)

❑ Vectors (3 values per sample):

- Accelerometer: [ax, ay, az] Measures linear acceleration (including gravity)
- Magnetometer: [mx, my, mz] Measures magnetic field (Earth's field)
- Gyroscope: [gx, gy, gz] Measures angular velocity (rotation speed)

❑ Point Clouds (N vectors):

- LIDAR, depth cameras.
- Challenge: Not fixed size. Variable number of points.

❑ Matrices (2D arrays):

- Image frames (640x480 pixels).
- Challenge: Megabytes per second. DMA required.

DMA: is a hardware mechanism that allows data to be transferred directly between devices and memory without involving the CPU

Sampling Theory - Why Time Matters?

❑ **The Setup:** You cannot capture what you do not see.

❑ **Nyquist-Shannon Theorem:**

➤ $f_s > 2 f_{\max}$

➤ f_s : Sampling frequency (Hz)

➤ $f_{\max}$: Highest frequency present in the signal

❑ **Rule of Thumb:**

➤ **2x** = Theoretical minimum.

➤ **5–10x** = Engineering reality.

❑ **Robotics Reality Check:**

"Your robot vibrates at 100 Hz. If you sample your IMU (Inertial Measurement Unit) at 100 Hz, you are measuring noise, not motion."

An IMU (Inertial Measurement Unit) is a device that measures a system's motion and orientation using a combination of sensors

Sampling Theory - Why Time Matters? (Cont.)

❑ What does “robot vibrates at 100 Hz” mean?

It means the mechanical vibration frequency is:

$$f_{vibration} = 100 \text{ Hz}$$

➤ So the robot completes **100 oscillation cycles per second**.

❑ What does “sample IMU at 100 Hz” mean?

Sampling at 100 Hz means:

$$f_s = 100 \text{ samples per second}$$

➤ So you take **one measurement every 0.01 seconds**:

$$T_s = \frac{1}{f_s} = \frac{1}{100} = 0.01 \text{ s}$$

Sampling Theory - Why Time Matters? (Cont.)

❑ What Actually Happens?

- If your vibration is: $x(t) = A\sin(2\pi \cdot 100t)$
- And you sample at: $t = 0, 0.01, 0.02, 0.03, \dots$
- Now check what happens: **At $t = 0.01$:**

$$2\pi \cdot 100 \cdot 0.01 = 2\pi$$

$$\sin(2\pi) = 0$$

- **At $t = 0.02$:**

$$2\pi \cdot 100 \cdot 0.02 = 4\pi$$

$$\sin(4\pi) = 0$$

- So every time you sample...
- You hit exactly the same phase of the wave.
- Your measurements look like: **0, 0, 0, 0, 0...** **It looks like no motion at all !!!**

Intuitive Example

❑ Flashing Light

- Imagine a light flashing **100** times per second at a fan spinning **100** times per second.
- The fan looks frozen.
- **It's moving — but you always see it at the same position !!!**

❑ The Wagon Wheel Effect:

- A wheel spinning forward at 110 Hz.
- Camera samples at 100 Hz.
- Wheel appears to spin **backward** at 10 Hz.
- **That's exactly what your IMU is doing.**

This is **Aliasing**

❑ Sampling at the same frequency as the vibration causes:

- The real motion to disappear
- Or to appear as a lower-frequency false signal
- Or to look like random noise

❑ This is called **aliasing**.

❑ **You are not measuring real vibration.**

❑ **You are measuring a distorted representation of it.**

Why It Becomes “Noise”

❑ In real systems:

- Vibration is **not** perfectly sinusoidal
- Frequency is **not** perfectly constant
- There are multiple vibration components

❑ So instead of clean zeros, you get:

- Irregular values
- High-frequency jitter
- What looks like random noise

But it is actually under-sampled vibration.

Robotics Interpretation (Very Important)

□ If:

- Motor vibration ≈ 100 Hz
- IMU sampling = 100 Hz

□ Then:

- You cannot distinguish vibration from real acceleration
- State estimation (Extended Kalman Filter (EKF), simultaneous localization and mapping (SLAM)) becomes corrupted
- The filter may interpret vibration as real motion

□ This leads to:

- Drift
- False pose updates
- Unstable control

Many robotics systems use filters like:

Extended Kalman Filter (EKF) → estimates robot's position, velocity, orientation

SLAM (Simultaneous Localization and Mapping) → estimates pose while building a map

These filters assume the input is mostly real motion, with some small noise.

If vibration appears in the sensor data:

The filter interprets it as actual movement

The filter updates the robot's pose and velocity incorrectly

Quantization – The Price of Digitization

❑ The Process:

Continuous voltage → Discrete integer.

❑ Resolution Examples:

Bit Depth	Steps	Step Size (5V ref)	CS Impact
8-bit	256	19.5 mV	noisy
10-bit	1024	4.88 mV	Common for basic sensors
12-bit	4096	1.22 mV	Good for precision
16-bit	65536	76 μ V	Overkill for most robotics

Step Size= Voltage Range (5) / $2^{**}bits$

Steps= $2^{**}bit_depth$

Sensor Taxonomy – Two Families

❑ **Proprioceptive** (Internal Sense):

Sensor	Rate	Bandwidth	CS Role
Encoder	High (kHz)	Low	Odometry, velocity
IMU	High (kHz)	Low	Orientation, impact
Current	Medium	Low	Stall detection

❑ **Exteroceptive** (External Sense):

Sensor	Rate	Bandwidth	CS Role
LIDAR	10–100 Hz	Medium	Mapping
Camera	30–120 Hz	High (MB/s)	Vision, ML
Microphone	44.1 kHz	High	Audio events

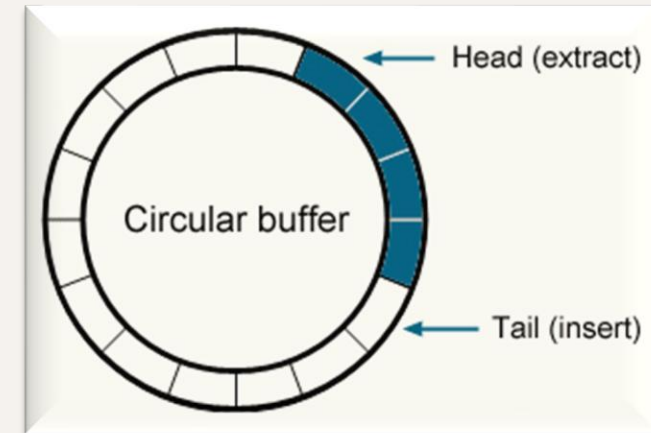
Buffering – Managing Asynchrony

□ The Problem:

- Sensor produces data at 1000 Hz.
- Main control loop runs at 100 Hz.
- *You cannot process every sample immediately.*
- **The Solution: Circular Buffer (Ring Buffer)**

□ Properties:

- Fixed memory size ($O(N)$).
- $O(1)$ insertion/removal.
- Old data overwritten automatically.



Real-Time Data Acquisition

Real-time systems must acquire sensor data within strict timing constraints. The choice of acquisition method affects:

- CPU load
- Timing predictability
- System responsiveness
- Power consumption

□ There are three main methods:

- Polling
- Interrupt-Driven I/O
- DMA (Direct Memory Access)

Polling

❑ Concept

The CPU repeatedly checks a device to see if new data is available.

❑ Characteristics

Aspect	Explanation
CPU Usage	High (CPU constantly checks)
Latency	Variable (depends on polling frequency)
Complexity	Low
Determinism	Weak

Determinism is "weak" in polling, it means you cannot guarantee or accurately predict the exact delay between a device having data ready and the CPU actually processing it. Determinism is "weak" in polling, it means you cannot guarantee or accurately predict the exact delay between a device having data ready and the CPU actually processing it.

Polling (Cont.)

❑ Advantages

- Very simple to implement
- Easy to debug
- No special hardware support required

❑ Disadvantages

- Wastes CPU cycles
- Can miss fast events
- Poor for high-speed systems

❑ Typical Use Cases

- Slow sensors (temperature, humidity)
- Simple microcontroller firmware
- Educational examples

Interrupt-Driven I/O

❑ Concept

- The device signals the CPU when data is ready.
- The CPU temporarily stops current execution and runs an **Interrupt Service Routine (ISR)**.

❑ Characteristics

Aspect	Explanation
CPU Usage	Medium
Latency	Low
Complexity	Medium
Determinism	Good (but ISR timing must be controlled)

Interrupt-Driven I/O (Cont.)

❑ Advantages

- Efficient CPU usage
- Good responsiveness
- Event-driven design

❑ Disadvantages

- More complex debugging
- Poorly designed Interrupt Service Routines (ISRs) can block the system
- Risk of priority inversion

❑ Typical Use Cases

- IMU sampling
- Wheel encoders
- Button presses
- Communication peripherals
(Universal Asynchronous Receiver-Transmitter UART)

DMA (Direct Memory Access)

❑ Concept

- Data is transferred directly between peripheral and memory **without** CPU involvement.
- The CPU only configures DMA once, and hardware handles the rest.

❑ Characteristics

Aspect	Explanation
CPU Usage	Almost zero during transfer
Latency	Deterministic
Complexity	High
Throughput	Very high

DMA (Cont.)

❑ Advantages

- Minimal CPU load
- Ideal for high data rates
- Predictable timing
- Enables real-time streaming

❑ Disadvantages

- Complex setup
- Hardware dependent
- Harder to debug

❑ Typical Use Cases

- Cameras
- High-speed ADC
- Audio streaming
- LiDAR
- High-rate IMU logging

When to Use What?

If you need...	Use
Simplicity	Polling
Event-driven behavior	Interrupt
High bandwidth & real-time streaming	DMA

❑ Robotics Perspective

In mobile robots:

- Encoders → **Interrupt**
- IMU at moderate speed (100–500 Hz) → **Interrupt**
- IMU at very high speed (>1 kHz) → **DMA**
- Camera → **DMA**
- Temperature sensor → **Polling**

Why Sensors Lie?

Real-world sensors never measure the true value directly.

Instead, we observe:

$$y(t) = x(t) + v(t)$$

Where:

$x(t)$ → True signal

$v(t)$ → Noise component

Different types of noise behave differently — and require different fixes.

Noise Taxonomy in Sensors

1. Gaussian (White) Noise:

Random variation around true value.

2. Salt-and-Pepper (Impulse) Noise:

Sudden spikes to max/min.

3. Drift (Bias Instability):

Sensor "zero" slowly changes.

Gaussian (White) Noise

❑ Definition

Random variations centered around the true value.

❑ Physical Sources

- Thermal noise (electrons moving randomly)
- ADC quantization noise
- Electronic circuit noise
- Sensor internal electronics

❑ Properties

- Random
- Zero mean

Gaussian (White) Noise - Example (IMU)

❑ If a stationary accelerometer reads: (9.78 , 9.83 , 9.80 , 9.79 , 9.81)

❑ That variation is Gaussian noise.

❑ Fix

➤ Because it is zero-mean:

Averaging works: $\bar{x} = \frac{1}{N} \sum x_i$

❑ Low-pass filtering works

Gaussian noise is typically high-frequency → Low-pass filters attenuate it.

Gaussian (White) Noise - Example (Zero Means)

□ 10 Measurements

Suppose the noise samples are:

Sample	Noise (n)	Measurement ($z = 5 + n$)
1	+0.10	5.10
2	-0.15	4.85
3	+0.05	5.05
4	-0.20	4.80
5	+0.08	5.08
6	-0.12	4.88
7	+0.03	5.03
8	-0.05	4.95
9	+0.07	5.07
10	+0.19	5.19

Gaussian (White) Noise - Example (Zero Means) Cont.

Step 1: Compute Mean of Noise

- **Sum of noise:**

$$0.10 - 0.15 + 0.05 - 0.20 + 0.08 - 0.12 + 0.03 - 0.05 + 0.07 + 0.19 = \mathbf{0.00}$$

- **Mean noise:** $\frac{0.00}{10} = 0$

- **Noise is zero mean.**

Step 2: Mean of Measurements

- **Sum of measurements:**

$$5.10 + 4.85 + 5.05 + 4.80 + 5.08 + 4.88 + 5.03 + 4.95 + 5.07 + 5.19 = \mathbf{50.00}$$

- **Mean measurement:** $\frac{50.00}{10} = 5.0$

- **The average measurement equals the true value.**

Gaussian (White) Noise - Example (Not Zero Means)

□ 10 Measurements

Suppose the noise samples are:

Sample	Noise (n)	Measurement ($z = 5 + n$)
1	+0.20	5.20
2	-0.05	4.95
3	+0.15	5.15
4	-0.10	4.90
5	+0.18	5.18
6	-0.02	4.98
7	+0.13	5.13
8	+0.05	5.05
9	+0.17	5.17
10	+0.29	5.29

Gaussian (White) Noise - Example (**Not Zero Means**) Cont.

Step 1: Compute Mean of Noise

➤ **Sum of noise:**

0.20, -0.05, 0.15, -0.10, 0.18, -0.02, 0.13, 0.05, 0.17, 0.29 = **1**

➤ **Mean noise:** $\frac{1}{10} = 0.1$

➤ **Noise is Not zero mean.**

Step 2: Mean of Measurements

➤ **Average measurement becomes: $5.0 + 0.1 = 5.1$**

➤ **The sensor now systematically overestimates distance.**

Salt-and-Pepper (Impulse) Noise

❑ Definition

Occasional extreme spikes:

- Maximum value
- Minimum value
- Very large outliers

❑ Physical Sources

- Loose wires
- Electromagnetic interference (EMI)
- Faulty ADC sample
- Dead camera pixel
- Power supply glitch

Salt-and-Pepper (Impulse) Noise - Example

- ❑ IMU readings: 9.80 , 9.82 , 47.00 (spike) , 9.79 , 9.81
- ❑ That 47 is not Gaussian. It is impulse noise.
- ❑ Why Averaging Fails?
 - If you average: $(9.8 + 9.82 + 47 + 9.79 + 9.81) / 5 = 17.244$
 - The result becomes completely wrong.
 - Impulse noise corrupts mean-based methods.

Salt-and-Pepper (Impulse) Noise – Example (Cont.)

❑ Fix: **Median Filter**

- Instead of averaging, take the median

Sorted: 9.79 , 9.80 , 9.81 , 9.82 , 47

Median = 9.81

- The spike disappears.

Drift (Bias Instability)

❑ Definition:

The long term drift of sensor measurement is bias instability which occurs even when the IMU is stationary. Such a small but continuous error accumulates and causes huge errors in navigation.

❑ Physical Sources

- Temperature changes
- MEMS (Micro-Electro-Mechanical Systems) aging
- Mechanical stress
- Power supply instability
- Gyroscope bias instability

Drift (Bias Instability) - Example

❑ **Robot is stationary, but gyro reads:**

- 0.01 rad/s
- 0.02 rad/s
- 0.03 rad/s
- 0.05 rad/s
- 0.08 rad/s

Over time this integrates into huge angle error !!!

Drift (Bias Instability)

❑ **Fix**

➤ **Periodic Recalibration**

Reset bias when robot is known to be stationary.

➤ **High-Pass Filtering**

Removes slow-changing components.

➤ **Sensor Fusion (Best Solution)**

Combine with another sensor:

- Gyro + Accelerometer (Complementary filter)
- IMU + GPS
- IMU + Vision

Practical Robotics Mapping

Sensor	Dominant Noise
IMU accelerometer	Gaussian
IMU gyroscope	Gaussian + Drift
Encoders	Quantization + Impulse
Camera	Gaussian + Salt & Pepper
Magnetometer	Drift + Disturbance

Encoding: Measures position or rotation of motors/wheels (counts ticks).
Used to estimate distance traveled or wheel rotation.

Statistical Model & The Estimation Problem

□ The Standard Measurement Model: $z_k = x_k + v_k$

Where:

- z_k : Measured value at time step k
- x_k : True state (unknown !!, what we want)
- v_k : Measurement noise

□ Assumptions: Zero-mean, Gaussian, Independent.

□ What This Equation Really Means?

- We never observe reality directly !!
- We observe: Truth + Uncertainty
- The measurement is a corrupted version of the true state.

Statistical Model & The Estimation Problem (Cont.)

□ The Engineer's Goal

- Given a stream of noisy measurements: $z_0, z_1, z_2, \dots, z_k$
- Produce the best possible estimate: $\hat{x}_k$
- **Such that:**
 - **Estimation error is minimized**
 - **Variance is reduced**
 - **Bias is controlled**
- This is **NOT** a coding problem or filtering trick.
- **This is a Statistical Estimation Problem**

Statistical Model & The Estimation Problem (Cont.)

❑ What Does “Best” Mean?

We must define an optimality criterion:

➤ Minimum Mean Square Error (MMSE)

measures the average squared difference between estimated (predicted) values and the actual value

➤ Maximum Likelihood (ML)

It assumes a specific probability distribution for the data, then uses calculus (often maximizing the log-likelihood) to find the parameters that best fit the data.

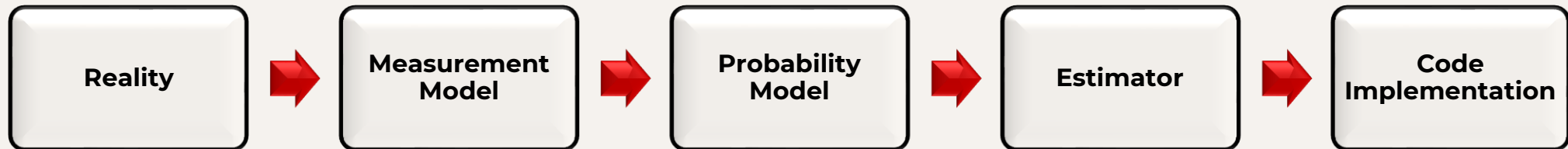
➤ Maximum A Posteriori (MAP)

It extends maximum likelihood estimation (MLE) by incorporating a prior distribution to improve estimates, especially with sparse data.

❑ **Before writing any code, we must answer: What are we optimizing?**

Statistical Model & The Estimation Problem (Cont.)

❑ Conceptual Flow



❑ Mathematics defines the estimator.

❑ Programming only implements it.

The math comes first. The code is just implementation.

Filter 1 – Moving Average

□ Core Idea (Intuition):

- **“Smooth by committee. Let the last N readings vote”**
- Instead of trusting one noisy measurement, we average the last N samples.
- Noise tends to cancel out and the real signal remains.
- **Mathematical Formula:**

$$\hat{x}_k = \frac{1}{N} \sum_{i=k-N+1}^k z_i$$

Where:

- z_i = Raw Sensor Measurement
- N = Window Size
- $\hat{x}_k$ = Filtered Output
- **It is simply the average of the last N measurements.**

Filter 1 – Moving Average (Cont.)

❑ Trade-Off

N Value	Effect
Large N	Very smooth, very slow response
Small N	Fast response, still noisy

❑ If N is large:

- Your robot reacts slowly
- Turns feel delayed
- Speed control feels soft

❑ This is especially noticeable in:

- PID controllers
- IMU filtering
- Go-to-goal behavior

Filter 1 – Moving Average (Cont.)

□ **Phase Lag** (Very Important in Robotics)

➤ If a **step input** happens at time $t = 0$:

➤ The output needs roughly:

$\frac{N}{2}$ *samples*

➤ To reach **50%** of the true value.

➤ **Meaning:** The filter introduces **delay**.

□ **Example (Numerical)**

- **Assume:** $N = 4$

Step input goes from $0 \rightarrow 10$

- **Samples:** 0, 0, 0, 10, 10, 10, 10

- **Moving average output:**

0, 0, 0, 2.5, 5, 7.5, 10

- **Notice:** It takes **~2 samples ($N/2 = 2$)** to reach **5** (**50% of 10**).

Filter 2 – Low-Pass Filter (Exponential)

- ❑ **The Intuition:** “Trust the new measurement a little. Trust the history a lot”.
- ❑ Instead of averaging many past samples, **we blend: The current measurement and The previous filtered value**
- ❑ **It's like a weighted compromise between past and present.**
- ❑ **The Formula (Discrete-Time)**

$$y_k = \alpha z_k + (1 - \alpha)y_{k-1}$$

Where:

- z_k = new sensor measurement
- y_{k-1} = previous filtered output
- $\alpha \in (0, 1)$ = smoothing factor
- y_k = new filtered output

Filter 2 – Low-Pass Filter (Exponential)

□ Understanding α (Alpha)

α Value	Behavior
$\alpha \approx 1$	Fast response, little smoothing, noisy
$\alpha \approx 0$	Very smooth, very slow response

□ Why It's Called “Exponential”

➤ If you expand the recursion:

$$y_k = \alpha z_k + \alpha(1 - \alpha)z_{k-1} + \alpha(1 - \alpha)^2 z_{k-2} + \dots$$

➤ **Older measurements get exponentially smaller weight.**

➤ That's why it's called: **Exponential Moving Average (EMA)**

Filter 3 – Median Filter

- ❑ **The Logic:** “Sort the last N readings. Take the middle one”.
- ❑ Instead of averaging values, we select the **middle value** after sorting.
- ❑ It does not compute a mean, It chooses a representative value.
- ❑ **The Problem with Average**
 - **One extreme value can destroy the result.**
- ❑ **Why Median Works**
 - **The median depends only on order, not magnitude.**
 - **Even if the spike becomes 10,000: Median stays the same.**
 - **That makes it extremely robust.**

Filter 4 – High-Pass Filter (HPF)

- ❑ Purpose: “Let changes through. Block constants”.
- ❑ A High-Pass Filter: Passes rapid changes, Removes constant (DC) components.
- ❑ Removing Gravity
 - Accelerometer measures: $a_{\text{total}} = a_{\text{motion}} + g$
 - Where:
 - a_{motion} = actual robot acceleration
 - g = gravity (constant $\approx 9.81 \text{ m/s}^2$)
 - If the robot tilts or moves, we want: a_{motion}
 - The high-pass filter removes the DC component (gravity).

Filter 4 – High-Pass Filter (Cont.)

❑ Discrete High-Pass Formula

❑ Derived from the low-pass filter:

$$y_k = \alpha(y_{k-1} + z_k - z_{k-1})$$

Where:

- z_k = current input
- z_{k-1} = previous input
- y_{k-1} = previous output
- $\alpha \in (0, 1)$

❑ Inside the parentheses we have:

$$z_k - z_{k-1}$$

This is simply:

- **Current value – Previous value**
- That is a discrete derivative.

❑ **If the signal:**

- **Is constant** → **difference = 0**
- **Is changing** → **difference ≠ 0**
- **So this term detects change.**

Filter 4 – High-Pass Filter (Cont.)

- ❑ Why Add y_{k-1} ?
- ❑ If we only used: $y_k = z_k - z_{k-1}$
- ❑ We would get a pure derivative:
 - Very noisy
 - Very unstable
- ❑ Instead, we accumulate previous output: $y_k = \alpha(y_{k-1} + \text{change})$
- ❑ This makes it: A smoothed derivative
- ❑ It keeps past high-frequency content but gradually decays.

- ❑ What Does α Do?

$$\alpha \in (0, 1)$$

- α close to 1 → keeps changes longer, stronger output
 - α small → changes decay quickly
- ❑ So α controls:
 - How long changes persist
 - How aggressive the filtering is

Filter 4 – High-Pass Filter (Cont.)

❑ What Happens for a Constant Signal?

- Suppose: $z_k = 5$ for all k
- Then: $z_k - z_{k-1} = 0$
- So: $y_k = \alpha y_{k-1}$
- The output decays toward zero.
- That means: Constant signals disappear.
- Exactly what a high-pass filter should do.

❑ What Happens for a Sudden Change?

- ❑ Suppose Input jumps: $5 \rightarrow 10$
- ❑ Then: $z_k - z_{k-1} = 5$
- ❑ So output becomes: $y_k = \alpha(y_{k-1} + 5)$
 - You get a spike.
- ❑ After that:
 - If input stays constant, The derivative becomes 0, Output decays.
- ❑ So HPF produces: A spike when something changes.

Filters Comparison

Feature	Moving Average	Exponential (EMA)	Median	High-Pass (HPF)
Main Idea	Average last N samples	Weighted blend of new + previous	Take middle of last N	Keep changes, remove constants
Strengths	Simple, smooths Gaussian noise	Fast, tunable, $O(1)$ memory	Removes spikes, robust to outliers	Removes DC offset, detects motion
Weaknesses	Large delay, sensitive to spikes	Spikes bleed through, requires tuning α	Higher computation, requires buffer	Amplifies spikes, noisy, cannot remove general noise
Best for Noise Type	Gaussian	Gaussian	Impulse / spikes	Constant offsets / drift
Memory	$O(N)$	$O(1)$	$O(N)$	$O(1)$
Typical Robotics Use	Slow sensors, temperature	IMU, wheel velocity, real-time control	LiDAR, ultrasonic, touch sensors, vision	Gravity removal, collision/impact detection, vibration detection

Calibration – Making Numbers Mean Something

□ Why Calibration Matters?

➤ Raw sensor readings are **not perfect numbers**:

- Encoders, IMUs, accelerometers, etc., may have **bias** and **scaling errors**.

- **If uncorrected, your robot thinks it moved more/less than it really did.**

□ Calibration makes sensor readings **physically meaningful**.

Calibration – Making Numbers Mean Something

1. Offset (Bias) Correction

- **Idea:** Sensor should read zero when it's stationary or at a known reference.
- **Example:**
 - **Robot stationary** → **accelerometer reads** $b = 0.05 \text{ m/s}^2$
 - **True** value should be **0**
- **Correction formula:**
$$x_{\text{cal}} = x_{\text{raw}} - b$$
- **Remove the constant bias.**

Calibration – Making Numbers Mean Something (Cont.)

2. Scale Factor Correction

➤ **Idea:** Sensor may not measure distances/angles accurately.

➤ **Example:**

- Robot moves **1.0 m**
- Encoder reports **0.95 m**
- Scale factor:

$$s = \frac{\text{true distance}}{\text{measured distance}} = \frac{1.0}{0.95} \approx 1.0526$$

➤ **Correction formula:**

$$x_{\text{cal}} = (x_{\text{raw}} - b) \cdot s$$

➤ **Combines bias removal and scaling.**

Calibration – Making Numbers Mean Something (Cont.)

Bias (Offset) Correction **Alone**

- Use if your sensor **always has a constant offset.**
- Example: Gyroscope reads **0.05°/s when stationary.**
- Effect: **Removes the fixed error.**
- Limitation:
- **If the sensor is also scaled incorrectly (reports 0.95 m when you moved 1.0 m), bias removal alone is not enough.**

Scale Factor Correction **Alone**

- Use if the sensor **is accurate at 0 but underestimates or overestimates proportionally.**
- Example: Wheel encoder reads **0.95 m for a true 1 m travel, but starts at 0 correctly.**
- Effect: **Fixes proportional error.**
- Limitation:
- **If there's a bias (starting offset), scaling alone will not fix it.**

Calibration – Making Numbers Mean Something (Cont.)

❑ Combined Correction (Bias + Scale)

- **Most robust approach:** first remove bias, then apply scale factor.
- **Formula:**

$$X_{cal} = (X_{raw} - b) \cdot s$$

- **Removes both constant offset and proportional error.**
- **Makes sensor readings accurate and meaningful.**

❑ Practical Rule in Robotics

- **If you want high-accuracy measurements, always use bias + scale correction together.**
- **Bias alone or scale alone is usually insufficient.**

Calibration – Making Numbers Mean Something (Cont.)

Example: Wheel Encoder Calibration

❑ Scenario

- Robot moves 1 m repeatedly
- Raw encoder readings: [0.95, 1.90, 2.85, 3.80] m
- True distances: [1, 2, 3, 4] m
- Observed bias at start: 0.05 m (sensor reads 0.05 m instead of 0)

❑ Bias-Only Correction

- $x_{cal} = x_{raw} - b$
- $b = 0.05$

Raw	Bias-Corrected	True
0.95	$0.95 - 0.05 = 0.90$	1
1.90	$1.90 - 0.05 = 1.85$	2
2.85	$2.85 - 0.05 = 2.80$	3
3.80	$3.80 - 0.05 = 3.75$	4

- ❑ Observation: Bias removed, but readings are still proportionally low → underestimates distance.

Calibration – Making Numbers Mean Something (Cont.)

□ Scale-Only Correction

➤ $x_{cal} = x_{raw} \cdot s$

➤ **Scale factor** $s = \frac{1.0}{0.95} \approx 1.0526$

Raw	Scale-Corrected	True
0.95	$0.95 * 1.0526 \approx 1.0$	1
1.90	$1.90 * 1.0526 \approx 2.0$	2
2.85	$2.85 * 1.0526 \approx 3.0$	3
3.80	$3.80 * 1.0526 \approx 3.997$	4

□ **Observation:** Scale fixes proportional error, but if there was a starting bias, first reading would be wrong.

Calibration – Making Numbers Mean Something (Cont.)

□ Bias + Scale Correction (Full Model)

➤ $x_{cal} = (x_{raw} - b) \cdot s$

➤ $b = 0.05, s = 1.0526$

Raw	Corrected	True
0.95	$(0.95 - 0.05) \cdot 1.0526 = 0.9 \cdot 1.0526 \approx 0.947$	1
1.90	$(1.90 - 0.05) \cdot 1.0526 = 1.85 \cdot 1.0526 \approx 1.946$	2
2.85	$(2.85 - 0.05) \cdot 1.0526 = 2.8 \cdot 1.0526 \approx 2.947$	3
3.80	$(3.80 - 0.05) \cdot 1.0526 = 3.75 \cdot 1.0526 \approx 3.947$	4

□ Observation: Both bias and proportional error are corrected → readings are closer to true distances.

□ Important Points:

- **Bias-only** → removes constant error, but proportional error remains
- **Scale-only** → fixes proportional error, but constant offset remains
- **Bias + Scale** → full correction → accurate, real-world measurement

Dynamic Calibration – Auto-zeroing

- ❑ **Concept:** “Calibrate while the robot is running, but only when you are sure it is still”.
- ❑ **Static calibration** is **one-time**, but sensors **drift over time** (temperature, voltage, mechanical wear).
- ❑ **Dynamic calibration** **updates** bias **automatically during operation**. **Often called auto-zeroing**.
- ❑ **Gyroscope Drift Example**
 - Gyro stationary reading: **0.1 rad/sec (should be 0)**
 - **Integrate over 10 s → 1.0 rad error**
 - **Robot thinks it has turned, but it hasn't**
 - **Problem:** Even small biases accumulate into **large orientation errors**.

Dynamic Calibration – Auto-zeroing (Cont.)

❑ Auto-Zero Logic Idea:

- Only update the bias when the robot is known to be still.
- Ignore updates when robot is moving (so you don't remove real motion)

❑ Example:

if robot_is_stationary: # e.g., wheel speeds ≈ 0 , no vibration

gyro_bias = moving_average(gyro_raw) # Update bias dynamically

else:

gyro_calibrated = gyro_raw - gyro_bias # Use last bias

❑ Danger: “If you calibrate while moving, you will calibrate out the actual motion”.

- The robot would ignore real turns
- Auto-zeroing must only run when motion = 0

Numerical Differentiation – The Noise Amplifier

The Problem

- You have **position measurements**: $x(t)$
- You want **velocity**:

$$v(t) = \frac{dx}{dt}$$

- **Discrete approximation:**

$$v_k = \frac{x_k - x_{k-1}}{\Delta t}$$

❑ Why This Is Dangerous

- Position sensor has **noise**: ± 1 mm
- Sampling rate: **100 Hz** ($\Delta t = 0.01$ s)

❑ Effect on velocity:

$$v_k = \frac{\text{noise}}{\Delta t} = \frac{0.001 \text{ m}}{0.01 \text{ s}} = 0.1 \text{ m/s}$$

- **Tiny position noise becomes huge velocity noise**
- **Robot thinks it is shaking violently, even when perfectly still.**

Dynamic Calibration – Auto-zeroing (Cont.)

❑ The Golden Rule:

“Never differentiate raw sensor data. **Filter first, then differentiate**”.

- Apply low-pass filter / moving average / EMA to position **before differentiating**
- **This removes high-frequency noise, preventing velocity spikes**

❑ Robotics Example

- **Wheel encoder readings** → **integrate to position** → **differentiate** → **velocity**
- If raw readings are noisy → **velocity estimate jumps** → **unstable control**
- **Solution:**
 - **Filter position (e.g., moving average)**
 - **Then apply numerical differentiation**

Sensor Fusion – Why One Isn't Enough

❑ The Limitations of Individual Sensors

Sensor	Good At	Bad At
Encoder	Precise over short distances	Slip, wheelie, skid
GPS	No drift, global accuracy	Low rate, multipath, denied environments
IMU	High rate, smooth readings	Drift, noise
Camera	Rich semantic information	High computation, sensitive to lighting

❑ Fusion Philosophy

“Use the strengths of each sensor to cover the weaknesses of others.”

❑ Example:

- ❑ Encoder → short-term precision
- ❑ IMU → smooth high-rate motion
- ❑ GPS → global reference
- ❑ Camera → semantic understanding

❑ By combining sensors, you can get:

- ❑ Accurate position over time
- ❑ Smooth and responsive motion estimates
- ❑ Robust perception in complex environments

The Complementary Filter

❑ The Problem: Tilt Estimation

Imagine your robot tilts slightly. You want to know its angle θ relative to vertical.

- **Accelerometer** → measures gravity → absolute tilt:
 - Pros: **Never drifts**
 - Cons: **Very noisy if robot vibrates**
- **Gyroscope** → measures angular velocity → integrate to get tilt:
 - Pros: **Smooth, responds fast**
 - Cons: **Drifts over time**
- Alone, **each sensor fails**:
 - **Accelerometer = shaky numbers**
 - **Gyroscope = slowly drifts away from reality**
- **Gyro friend** → “I’m really fast at detecting small changes.”
- **Accel friend** → “I’m accurate over the long term, but shaky.”
- **Solution:** Listen mostly to the gyro for fast changes, but occasionally correct with the accelerometer.

The Complementary Filter (Cont.)

□ How the Filter Works (Intuition)

$$\theta_k = \alpha(\theta_{k-1} + \omega\Delta t) + (1 - \alpha)\theta_{accel}$$

- $\theta_{k-1} + \omega\Delta t \rightarrow$ gyro estimate
- $\theta_{accel} \rightarrow$ accelerometer estimate
- $\alpha \approx 0.97 \rightarrow$ 97% gyro trust, 3% accel correction

□ Step-by-Step Example

Assume:

- Sampling rate: 100 Hz ($\Delta t = 0.01$ s)
- Previous tilt: $\theta_{k-1} = 10^\circ$
- Gyro reads: $\omega = 2^\circ/\text{s} \rightarrow$ small rotation in this timestep
- Accelerometer reads: $\theta_{accel} = 11^\circ \rightarrow$ noisy measurement

The Complementary Filter (Cont.)

❑ Step 1: Gyro prediction

$$\theta_{gyro} = \theta_{k-1} + \omega \Delta t = 10 + 2 * 0.01 = 10.02^\circ$$

❑ Step 2: Combine with accelerometer

$$\begin{aligned}\theta_k &= 0.97 * 10.02 + 0.03 * 11 \\ &= 10.0294 + 0.33 = 10.3594^\circ \approx 10.36^\circ\end{aligned}$$

❑ Result:

- Mostly trusts gyro → smooth
- Small correction from accelerometer → prevents long-term drift

❑ Complementary filter → fuses the two signals

❑ Smooth, follows real tilt, doesn't drift, not too noisy

❑ Gyro → fast, noise-free, but can slowly wander

❑ Accelerometer → slow, accurate, but noisy

❑ Complementary filter → “best of both worlds”: mostly gyro + small accel corrections

Outlier Detection – The “Glitch” Filter

□ Core Principle

“Physics is a constraint. Use it.”

- **Sensors can lie, Physics does not.**
- Instead of blindly trusting numbers, ask:

“Is this measurement physically possible?”

If not → reject it.

1. Rate Limit Check (Speed Constraint)

- **Idea: Objects cannot teleport.**

If your robot moves at maximum **1 m/s**, and your sampling rate is **100 Hz (dt = 0.01 s)**, Then the maximum position change per step is:

$$\Delta x_{max} = v_{max} \cdot dt = 1 \cdot 0.01 = 0.01 \text{ m}$$

That's 1 cm per sample.

Example (Glitch)

- **Current position: 2.5**
- **Previous Position: 2.0**
- **Change (Δx) = 0.5**
- **But physical limit is: 0.01**
- **Impossible:** Your robot did not teleport **50 cm** in **0.01 s**.
→ **Reject the reading.**

Outlier Detection – The “Glitch” Filter (Cont.)

2. Value Limit Check (Range Constraint)

Idea: **Some values are physically impossible.**

Examples:

Sensor	Impossible Value
Distance sensor	Negative distance
Temperature	1000°C indoors
Battery voltage	200V in small robot

Outlier Detection – The “Glitch” Filter (Cont.)

3. Variance Check (Dead Sensor Detection)

- Idea: **Real-world signals always vary slightly.**
- If a sensor outputs exactly the same number for 1000 samples, **something is wrong.**
- Fact: **Physics has noise.**
- **Perfect constancy = suspicious.**

Outlier Detection – The “Glitch” Filter (Cont.)

Why This Matters?

❑ **Without glitch filtering:**

- One bad encoder spike → robot thinks it moved 5 meters
- One IMU spike → controller destabilizes
- One GPS jump → path planner goes crazy

❑ **With glitch filtering:**

- Robot stays stable
- Controllers remain safe
- Estimators stay realistic

Sensor Diagnostics – Detecting Death

- ❑ **Core Question:** How does software know a sensor has failed?
- ❑ Sensors don't send a message saying: "Hi, I'm broken."
- ❑ Software must **detect abnormal patterns**.

1. The "Stuck" Detector (Frozen Sensor)

- **Idea:** Real-world signals always fluctuate slightly.
- **Even if a robot is stationary:**
 - IMU has noise
 - Encoders jitter slightly
 - LIDAR distances vary a little
- **If variance $\approx 0 \rightarrow$ suspicious.**

Sensor Diagnostics – Detecting Death (Cont.)

2. The “Saturation” Detector (Clipped Sensor)

- Idea: **Sensors have physical limits.**
- Example:
 - ADC max value = 1023
 - LIDAR max range = 12 m
 - IMU max acceleration = $\pm 16g$
- **If sensor outputs maximum value repeatedly, it may be saturated.**

3. The “Noise Explosion” Detector (Broken Sensor)

- Idea: **All sensors have expected noise levels.**
- If variance **suddenly** becomes huge:
 - **Loose wire**
 - **EMI interference**
 - **Damaged hardware**



Any
Questions